

Resource Adaptivity at Task-Level

Jonas Posner

`jonas.posner@uni-kassel.de`

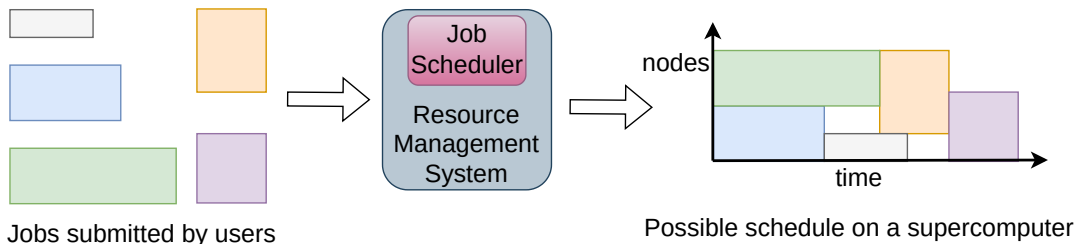
The 7th Annual Parallel Applications Workshop, Alternatives To MPI+X
(PAW-ATM), colocated with SC24

Atlanta, GA, U.S.

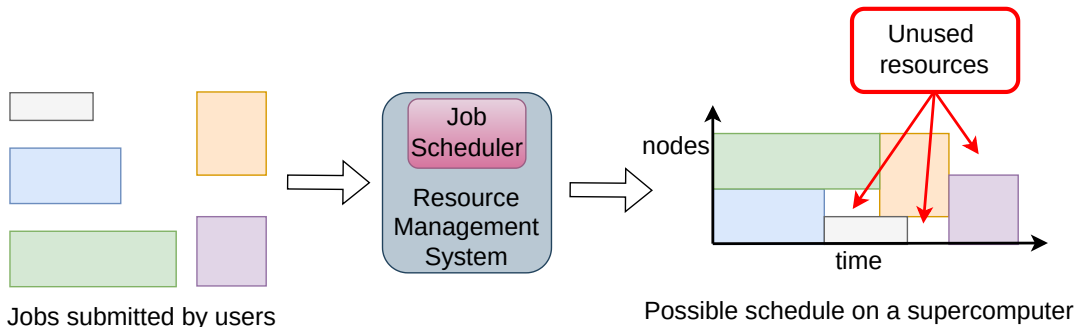


17th November 2024

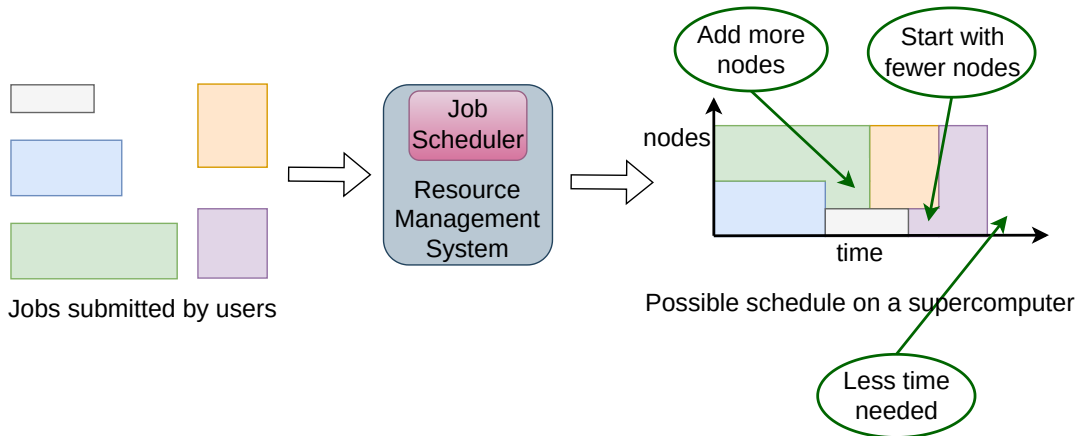
Motivation: Job Scheduling on Supercomputers



Motivation: Job Scheduling on Supercomputers



Motivation: Job Scheduling on Supercomputers



Motivation: Rigid Programs

Rigid Programs

- State of the art
- Users define *exact* number of nodes at job submission
- Resource manager assigns jobs to nodes and these assignments do not change

Motivation: Rigid Programs vs. Adaptive Programs

Rigid Programs

- State of the art
- Users define *exact* number of nodes at job submission
- Resource manager assigns jobs to nodes and these assignments do not change

Adaptive Programs

- Research
- Users may define *minimum* and *maximum* number of nodes
- Nodes can be added/removed at runtime
- *Malleable*: Resource managers initiate resource changes
- *Evolving*: Programs initiate resource changes themselves

Why are we not currently using adaptive jobs?

- Support in resource managers (Slurm etc.)
 - Requires interactions between resource manager and programs
- Support in programming systems (MPI etc.)
 - Only limited adaptivity functionalities
- Programming adaptive applications
 - Requires significantly increased programming complexity

Why are we not currently using adaptive jobs?

- Support in resource managers (Slurm etc.)
 - Requires interactions between resource manager and programs
- Support in programming systems (MPI etc.)
 - Only limited adaptivity functionalities
- Programming adaptive applications
 - Requires significantly increased programming complexity



This work!
(in a prototypical setting)

Background: Asynchronous Many-Task (AMT)

- *Alternative programming model to MPI*
- Computations are split into many small *tasks*
- The AMT runtime system dynamically assigns tasks to *workers*
- Enhancing programmer productivity
- AMT's transparent resource management is ideal for:
 - Load balancing for dynamic and irregular workloads, and
 - **Resource adaptivity**
- *There is still a lack of AMTs providing adaptivity in an uncomplicated and effective way*

Contributions

- Adaptivity extension to the “*APGAS for Java*” AMT
- Programmers enable adaptivity by specifying actions required when adding/removing nodes at runtime
- Runtime changes, including process initiation and termination, are managed automatically
- *Malleable* programs react to externally initiated resource changes
- *Evolving* programs automatically initiate resource changes themselves in response to their computational loads at runtime
- Resource changes do *not* interrupt the computations

Contributions

- Practical usability demonstration
 - Adaption of a Load Balancing library and its benchmarks
 - *No* additional programming effort required for resource adaptivity
- Generic communication interface: programs $\leftrightarrow$ resource managers
 - Communication is not restricted to any fixed points
- Prototype resource manager
 - Five job scheduling algorithms supporting rigid, malleable, and evolving jobs

Evaluation

- Real-world experiments
- Add/Remove one node:
 - Moving tasks/data: 0.5 sec
 - Starting/terminating process: 3.5 sec
- Job batches: 100% *malleable* / *evolving* jobs:
 - 13% / 23% decrease in job batch makespan
 - 20% / 4% increase in node utilization
 - 3% / 29% decrease in job turnaround time

Conclusions

- Both malleable and evolving capabilities can be effectively implemented in AMT
- **Adaptive programs have a huge potential to increase supercomputer efficiency**
- This work is part of a bigger effort on resource adaptivity
 - *Project Wagumo*: <https://github.com/projectwagomu>
- We are currently working on:
 - Implementing a new AMT exploiting new adaptive MPI Sessions
 - Simulating real supercomputer log traces but with adaptivity

Related Publications

1. Patrick Finnerty, Jonas Posner, Janek Bürger, Leo Takaoka, and Takuma Kanzaki. *“On the Performance of Malleable APGAS Programs and Batch Job Schedulers”*. In: Springer Nature Computer Science. 2024.
<https://doi.org/10.1007/s42979-024-02641-7>
2. Jonas Posner, Raoul Goebel, and Patrick Finnerty. *“Evolving APGAS Programs: Automatic and Transparent Resource Adjustments at Runtime”*. In: Proceedings Workshop on Asynchronous Many-Task Systems and Applications (WAMTA). 2024. https://doi.org/10.1007/978-3-031-61763-8_15
3. Jonas Posner. *“The Impact of Evolving APGAS Programs on HPC Clusters”*. In: Proceedings Euro-Par Parallel Processing Workshops (DynResHPC). 2024. To appear. <https://easychair.org/publications/preprint/7Rwf>

Thank you for your attention!

Please feel free to ask questions!